

Course Title: Data Analysis using R Programming Course code: CS1602-1
Semester: III Section: C Faculty: Dr. Anisha P Rodrigues
YEAR :2023-24

1. Create R program to write a user defined functions and perform the following operations on user input.
 - a. Check the number is even or odd
 - b. Print squares of numbers in sequence.
 - c. Find the factors of a given number.
 - d. Create vector of integers and sort them in ascending/descending order.

Solution:

- a. Check the number is even or odd

```

even_odd <-function(a )      #Function Definition
{
#of lines to be scanned
  if(a %% 2 == 0)
  {
    print("The number is Even Number")
  }
  else
  {
    print("The number is Odd Number")
  }
}
print("Enter the number to be checked")
a <-scan(nlines=1)          #Scan function allows to read and nlines allows number
even_odd(a)                  #Function Call

```

- b. Print squares of numbers in sequence.**

```
sqr <- function (n)
{
  print ("The Square of Numbers is:")
  for(i in 0:n)
  print(i^2)
}
```

```
print("Enter the Range:")
n <- scan(nlines=1)
sqr(n)
```

- c. Find the factors of a given number.

```
print_factors = function(n) {
  cat("The factors of",n,"are:")
  for(i in 1:n) {
    if((n %% i) == 0) {
```

```

    print(i)
  }
}
}

```

```

print("Enter the Number:")
n <- scan(nlines=1)
print_factors(n)

```

d. Create vector of integers and sort them in ascending/descending order.

```

srt<-function(a){
  v<-sort(a, decreasing = TRUE)
  print("DESCENDING ORDER")
  print(v)

  x<-sort(a, decreasing = FALSE)
  print("ASCENDING ORDER")
  print(x)
}
a <-scan(nlines=6)    #Scan function allows to read and nlines allows number
srt(a)                #Function Call

```

2. Write a R program to create a list containing a vector, a matrix and a list and give names to the elements in the list.
 - a. Count number of objects in a given list and Find the length of the first vector of a given list.
 - b. Access the first and third element of the list and extract all elements except the third element of the first vector of a given list.
 - c. Add element at the end of the list and Update the third element of the list and remove the last element.
 - d. Merge a list of vector to original list, convert a second and last element of a given list into vector and add these two vectors.

Solution:

a. Count number of objects in a given list and Find the length of the first vector of a given list.

```

list_data <- list(c("Red","Green","Black","Orange","Yellow"), matrix(c(1,3,5,7,9,11), nrow = 2),
  list("Python", "PHP", "Java"))
print("List:")
print(list_data)

names(list_data) = c("Color", "Odd numbers", "Languages")
print("List with column names:")
print(list_data)

print("Number of objects in the said list:")
length(list_data)

```



```
print("Length of the vector 'Color' of the said list")
print(length(list_data$Color))
```

b. Access the first and third element of the list and extract all elements except the third element of the first vector of a given list.

```
print('1st element:')
print(list_data[1])
print('3rd element:')
print(list_data[3])
print("First vector without third element:")
list_data$Color = list_data$Color[-3]
print(list_data$Color)
```

c. Add element at the end of the list and Update the third element of the list and remove the last element.

```
print("Add a new element at the end of the list:")
list_data[4] = "New element"
print("New list:")
print(list_data)
```

```
print("Update the third element of the list:")
list_data$Languages[4] = "R programming"
print("New list:")
print(list_data)
```

```
print("Remove the last element of the list:")
list_data[4] = NULL
print("New list:")
print(list_data)
```

d. Merge list of vectors to original list, convert a second and last element of a given list into vector and add these two vectors.

```
n1 = list(c(1,2,3,5,6,7))
print("Merge the said lists:")
m1 = c(list_data, n1)
print("New merged list:")
print(m1)
print(m1[2])
print(m1[4])
print("Convert the lists to vectors:")
v1 = unlist(m1[2], use.names = FALSE)
v2 = unlist(m1[4])
print(v1)
print(v2)
print("Add two vectors:")
v = v1 + v2
print("New vector:")
print(v)
```

3. Create matrices in R and perform the following operations on them.
- Addition, Subtraction of two matrices and product of matrix and its transpose.
 - Column sum, Mean across rows, Total Sum of a matrix and Sort the matrix elements across columns in ascending order.
 - Inverse of the matrix.
 - Find row and column index of maximum and minimum value in a given matrix.
 - Addition, Subtraction of two matrices and product of matrix and its transpose.

Solution:

a. Addition, Subtraction of two matrices and product of matrix and its transpose.

```
# Create two 3x3 matrices.  
matrix1 <- matrix(c(2,1,1,1,1,-1,1,1,2), nrow=3,ncol=3)  
print(matrix1)  
matrix2 <- matrix(c(5, 2, 0, 9, 3, 4,2,3,2), nrow=3,ncol=3)  
print(matrix2)
```

```
# Add the matrices.  
result <- matrix1 + matrix2  
cat("Result of addition","\n")  
print(result)
```

```
# Subtract the matrices  
result <- matrix1 - matrix2  
cat("Result of subtraction","\n")  
print(result)  
# Product of matrix and its transpose.  
result <- matrix1 %*% t(matrix1)  
cat("Result of product of matrix and its transpose","\n")  
print(result)
```

b. Column sum, Mean across rows, Total Sum of a matrix and Sort the matrix elements across columns in ascending order.

```
#Column Sum  
print("Column sum:")  
colSums(matrix1)  
print("Mean across rows:")  
apply( matrix1, 1, mean)  
print("Total sum:")  
sum(matrix1)  
print("Matrix elements in sorted order column wise:")  
apply(matrix1, 2, sort)
```

c. Inverse of the matrix.

```
#Inverse of the matrix  
invn=solve(matrix1)  
print(invn)
```



d. Find row and column index of maximum and minimum value in a given matrix

```
matrix2 <- matrix(c(5, 2, 0, 9, 3, 4,2,3,2), nrow=3,ncol=3)
print(matrix2)
```

```
result = which(matrix2 == max(matrix2), arr.ind=TRUE)
print("Row and column of maximum value of the said matrix:")
print(result)
result = which(matrix2 == min(matrix2), arr.ind=TRUE)
print("Row and column of minimum value of the said matrix:")
print(result)
```

4. Create R program and perform the following operations on them.

- Take user name as input string, display the number characters present in the string, convert the string into uppercase and display the middle character of the string.
- Create function called `is_palindrome()` that determines whether or not a given string is a palindrome. The function should take a single parameter.
- Create an ordered factor of 20 alternating pairs of 1s and 2s, label the data with the "ON" and "OFF".

Solution:

- a. Take user name as input string, display the number characters present in the string, convert the string into uppercase and display the middle character of the string.**

```
midf<-function(str){
  print(nchar(str))
  print(toupper(str))
  n1<-nchar(str)+1
  mc<-substring(str,n1%%2,(n1+1)%2)
  print(mc)
}
name<- readline("Enter your name:")
midf(name)
```

- b. Create function called `is_palindrome()` that determines whether or not a given string is a palindrome. The function should take a single parameter.**

```
is_palindrome <- function(x){
  a <- substring(x,seq(1,nchar(x)) , seq(1,nchar(x)))
  paste(rev(a),sep="",collapse="") == paste(a,sep="",collapse="")
}
str<- readline("Enter string:")
print(is_palindrome(str))
```

- c. Create an ordered factor of 20 alternating pairs of 1s and 2s, label the data with the "ON" and "OFF".**

```
gl(2, 2, 20,labels = c("ON", "OFF"),ordered=TRUE)
```

5. Perform the following operations on Data Frame

- a. Create the first data frame that contains student Details consisting of Student ID, and marks for three subjects, namely, English, Maths, and Science. Create the second data frame with column Student ID and State. Join both the data frames on Student ID.
- b. Add the new subject 'Social' to joined data frame of (a) using cbind() function and add 2 more student details using rbind() function. Get the Student ID and state details.
- c. Use the data frame of (b) and organize the data with respect to Student ID and State using the melt() function. Use the molten data and reshape the data to its original form using cast() function.

Solution:

- a. **Create the first data frame that contains student Details consisting of Student ID, and marks for three subjects, namely, English, Maths, and Science. Create the second data frame with column Student ID and State. Join both the data frames on Student ID.**

```
df1      =      data.frame(StudentId      =      c(101:106),English=c(35,65,80,40,79,91),
Maths=c(40,50,70,35,56,89),Science=c(38,69,87,45,68,95))
print(df1)
df2 = data.frame(StudentId = c(101,102,104,106,107,108),State = c("Karnataka","Kerala",
"Karnataka","Andra Pradesh", " Maharashtra", "Delhi"))
print(df2)
df = merge(x = df1, y = df2, by = "StudentId")
print(df)
```

- b. **Add the new subject 'Social' to joined data frame of (a) using cbind() function and add 2 more student details using rbind() function. Get the Student ID and state details and Get the maximum salary. Get the student details belongs to "Karnataka" State and Maths marks greater than 50.**

```
df=cbind(df, Social=c(46,67,78,70))
print(df)
# Create the second data frame
stu.newdata =
data.frame(StudentId=c(107,108),English=c(70,86),Maths=c(68,84),Science=c(63,83),Social=c(52,
73),State = c("Karnataka","Kerala"))
# Bind the two data frames.
stu.finaldata = rbind(df, stu.newdata)
print(stu.finaldata)

result <- data.frame(stu.finaldata$StudentId, stu.finaldata$State)
print(result)
```

- c. **Use the data frame of (b) and organize the data with respect to Student ID and State using the melt() function. Use the molten data and reshape the data to its original form using cast() function.**

```
install.packages("reshape")
```



```
#Loading the libraries
library(reshape)
# Original data frame
cat("Original data frame:\n")
print(df)

# Organize data w.r.t. StudentId and State
molten.data <- melt(stu.finaldata, id = c("StudentId", "State"), variable = "Subjects")

cat("\nAfter melting data frame:\n")
print(molten.data)

# Reshaped data
cast.data <- cast(molten.data, StudentId+State~Subjects)
print(cast.data)
```

6. Create a Data Frame in R for maintaining Employee Details using .csv file that contains 5 Employee Details consisting of Employee ID, Employee name, Salary, and Joining date and perform the following analysis on it.
 - a. Get the structure of the data frame.
 - b. Add the Department column to existing data frame using cbind() function and add 2 employees details rbind() function.
 - c. Get the employee's name and salary details and Get the maximum salary.
 - d. Get the persons in IT department whose salary is greater than 600 and Get the employee name who joined on or after 2014.

Solution:

- a. **Get the structure of the data frame and get the maximum salary**

```
# Create the data frame.
emp.data <- read.csv("input.csv")
print(emp.data)
str(emp.data)
```

- b. **Add the Department column to existing data frame using cbind() function and add 2 employees details rbind() function.**

```
emp.data<-cbind(emp.data, dept=c("IT","Operations","IT","HR","Finance"))
print(emp.data)
```

```
# Create the second data frame
emp.newdata <- data.frame(
id = c (6:8),
name = c("Rasmi","Pranab","Tusar"),
salary = c(578.0,722.5,632.8),
start_date = c("2013-05-21","2013-07-30","2014-06-17"),
dept = c("IT","Operations","Finance"))
```

```
# Bind the two data frames.
```

```
emp.finaldata <- rbind(emp.data,emp.newdata)
print(emp.finaldata)
```

c. **Get the employee name and salary details and Get the maximum salary.**

```
# Get the max salary from data frame.
sal <- max(emp.finaldata$salary)
print(sal)
result <- data.frame(emp.finaldata $emp_name, emp.finaldata$salary)
print(result)
# Organize data w.r.t. StudentId and State
```

d. **Get the persons in IT department whose salary is greater than 600 and Get the employee name who joined on or after 2014.**

```
info <- subset(emp.finaldata, salary > 600 & dept == "IT")
print(info)
```

```
retval <- subset(emp.finaldata, as.Date(start_date) > as.Date("2014-01-01"))
print(retval)
```

7. Create a R data frame to manage Student Scores using a CSV file and perform following analysis tasks:

- a. Write an R function named `read_csv_data` that takes a file name as an argument and returns a data frame containing the data from the CSV file. Create a function to find and display the student with the highest total score (Math_Score + Science_Score + English_Score) from the loaded data.
- b. Write an R function named `calculate_pass_percentage` that takes a data frame and a passing threshold as arguments and returns the percentage of students who passed all subjects (scores above the threshold).
- c. Use the `dplyr` library to write a function named `calculate_subject_averages` that calculates the average scores for each subject (Math, Science, English) and returns them as a data frame.
- d. Create a function named `save_passed_students` that takes a data frame and a passing threshold as arguments. The function should save a new CSV file named `passed_students.csv` containing only the rows of students who passed all subjects with scores above the threshold.

Solution:

- a. **Write an R function named `read_csv_data` that takes a file name as an argument and returns a data frame containing the data from the CSV file. Create a function to find and display the student with the highest total score (Math_Score + Science_Score + English_Score) from the loaded data**

```
read_csv_data <- function(file_name) {
  data <- read.csv(file_name)
  return(data)
}
```



```
# Function to read CSV data
data <- read_csv_data("student_scores.csv")
```

```
find_highest_total_score <- function(data) {
total_scores <- data$Math_Score + data$Science_Score + data$English_Score
max_index <- which.max(total_scores)
highest_scoring_student <- data[max_index, "Name"]
return(highest_scoring_student)
}
```

```
# Function to find and display the student with the highest total score
highest_scorer <- find_highest_total_score(data)
cat("Student with the highest total score:", highest_scorer, "\n")
```

b. Write an R function named `calculate_pass_percentage` that takes a data frame and a passing threshold as arguments and returns the percentage of students who passed all subjects (scores above the threshold).

```
# Get the max salary from data frame.
calculate_pass_percentage <- function(data, threshold) {
passing_students <- data[data$Math_Score >= threshold & data$Science_Score >= threshold &
  data$English_Score >= threshold, ]
pass_percentage <- (nrow(passing_students) / nrow(data)) * 100
return(pass_percentage)
}
# Function to calculate pass percentage
pass_percentage <- calculate_pass_percentage(data, 70) # Assuming the pass threshold is 70
cat("Pass Percentage:", pass_percentage, "%\n")
```

c. Use the `dplyr` library to write a function named `calculate_subject_averages` that calculates the average scores for each subject (Math, Science, English) and returns them as a data frame.

```
install.packages("dplyr")
library(dplyr)

calculate_subject_averages <- function(data) {
subject_averages <- data %>%
summarize(
  Avg_Math = mean(Math_Score),
  Avg_Science = mean(Science_Score),
  Avg_English = mean(English_Score)
)
return(subject_averages)
}

# Function to calculate subject averages
subject_averages <- calculate_subject_averages(data)
print("Subject Averages:")
print(subject_averages)
```

```
#summarize_at(c('Math_Score','Science_Score', 'English_Score'),mean
```

d. Create a function named `save_passed_students` that takes a data frame and a passing threshold as arguments. The function should save a new CSV file named `passed_students.csv` containing only the rows of students who passed all subjects with scores above the threshold.

```
library(dplyr)
save_passed_students <- function(data, threshold) {
  passing_students <- data %>%
  filter(Math_Score >= threshold, Science_Score >= threshold, English_Score >= threshold)
  write.csv(passing_students, "passed_students.csv", row.names = FALSE)
}
# Function to save passed students to a new CSV file
save_passed_students(data, 70) # Assuming the pass threshold is 70
cat("Passed students have been saved to 'passed_students.csv'\n")
```

8. Write an R program that accomplishes the following tasks by implementing user-defined functions:
 - a. Define a function to calculate the age of a person given their birthdate. The function should take the birthdate as input and return the age in years.
 - b. Create a function that calculates the number of days between two given dates. The function should accept two dates as inputs and return the difference in days.
 - c. Implement a function that takes a date and time in the format "YYYY-MM-DD HH:MM:SS" and converts it into a more user-friendly format, displaying the day of the week, month, day, and year along with the time in "Hour:Minute AM/PM" format.
 - d. Create a function that receives a date and time in the format "YYYY-MM-DD HH:MM:SS" and returns the day of the year and the week number of the year.

Solution:

- a. **Define a function to calculate the age of a person given their birthdate. The function should take the birthdate as input and return the age in years.**

```
# Define a function to calculate age
calculate_age <- function(birthdate) {
  # Get the current date
  current_date <- Sys.Date()
  # Calculate the age in years
  age <- as.integer(difftime(current_date, birthdate, units = "days") / 365.25)
  return(age)
}
# Example usage
birthdate <- as.Date("1990-05-15")
age <- calculate_age(birthdate)
cat("Age:", age, "years")
```



- b. **Create a function that calculates the number of days between two given dates. The function should accept two dates as inputs and return the difference in days.**

```
# Define a function to calculate days between two dates
calculate_days_between <- function(date1, date2) {
  # Calculate the difference in days
  days_difference <- as.integer(difftime(date2, date1, units = "days"))
  return(days_difference)
}
# Example usage
date1 <- as.Date("2023-01-15")
date2 <- as.Date("2023-08-18")
days_diff <- calculate_days_between(date1, date2)
cat("Days between the two dates:", days_diff, "days")
```

- c. **Implement a function that takes a date and time in the format "YYYY-MM-DD HH:MM:SS" and converts it into a more user-friendly format, displaying the day of the week, month, day, and year along with the time in "Hour:Minute AM/PM" format.**

```
# Define a function to convert date and time format
convert_datetime_format <- function(datetime_string) {
  # Convert string to POSIXct object
  datetime <- as.POSIXct(datetime_string, format = "%Y-%m-%d %H:%M:%S")
  # Format components
  formatted_date <- format(datetime, format = "%A, %B %d, %Y")
  formatted_time <- format(datetime, format = "%I:%M %p")
  # Combine and return the formatted date and time
  formatted_datetime <- paste(formatted_date, formatted_time)
  return(formatted_datetime)
}
# Example usage
datetime_string <- "2023-08-18 14:30:00"
formatted_datetime <- convert_datetime_format(datetime_string)
cat("Formatted date and time:", formatted_datetime)
```

- d. **Create a function that receives a date and time in the format "YYYY-MM-DD HH:MM:SS" and returns the day of the year and the week number of the year.**

```
# Define a function to extract day of the year and week number
extract_day_and_week <- function(datetime_string) {
  # Convert string to POSIXct object
  datetime <- as.POSIXct(datetime_string, format = "%Y-%m-%d %H:%M:%S")
  # Extract day of the year and week number
  day_of_year <- as.numeric(format(datetime, format = "%j"))
  week_number <- as.numeric(format(datetime, format = "%U"))
  return(list(day_of_year = day_of_year, week_number = week_number))
}
```

```
# Example usage
datetime_string <- "2023-08-18 14:30:00"
result <- extract_day_and_week(datetime_string)
cat("Day of the year:", result$day_of_year, "\n")
cat("Week number of the year:", result$week_number)
```

9. Consider the data, `X <- c(21, 62, 10, 53)` and `Labels <- c("London", "New York", "Singapore", "Mumbai")`. Write the R program to create the following Pie chart
- Create a simple Pie chart with Labels that describes the slices.
 - Create a chart with rainbow colors and add an appropriate title.
 - Create a 3-D chart with legend and slice percentages and save that chart

Solution:

- a. Create a simple Pie chart with Labels that describes the slices.**

```
# Create data for the graph.
x <- c(21, 62, 10, 53)
labels <- c("London", "New York", "Singapore", "Mumbai")
# Plot the chart. pie(x,labels)
```

- b. Create a chart with rainbow colors and add an appropriate title.**

```
# Create data for the graph.
x <- c(21, 62, 10, 53)
labels <- c("London", "New York", "Singapore", "Mumbai")
# Plot the chart with title and rainbow color pallet.
pie(x, labels, main = "City pie chart", col = rainbow(length(x)))
```

- b. Create a 3-D chart with legend and slice percentages and save that chart.**

```
# Get the library.
library(plotrix)
# Create data for the graph.
x <- c(21, 62, 10, 53)
labels <- c("London", "NewYork", "Singapore", "Mumbai")
piepercent<- round(100*x/sum(x), 1)
# Give the chart file a name.
png(file = "city_percentage_legends.jpg")
# Plot the chart.
pie3D(x, labels = piepercent, main = "City pie chart", col =rainbow(length(x)))
legend("topright", c("London", "NewYork", "Singapore", "Mumbai"), cex =
0.8, fill = rainbow(length(x)))
# Save the file.
dev.off()
```

10. Write the R program to plot the following charts

- Create the Bar chart for `H <- c(7,12,28,3,41)` give the chart file a name and plot the chart.
- Plot the line chart for the data in (a) and save that chart
- Consider the data `v <- c(19, 23, 11, 5, 16, 21, 32,14, 19, 27, 39)`. Create a simple histogram for Range of X and Y values.
- Create a histogram using non-uniform width for the vector v.



Solution:

a. Create the Bar chart for $H \leftarrow c(7,12,28,3,41)$ give the chart file a name and plot the chart.

```
# Create the data for the chart
H <- c(7,12,28,3,41)
# Give the chart file a name
png(file = "barchart.png")
# Plot the bar chart
barplot(H)
b) Save the file
# Save the file
dev.off()
```

b. Plot the line chart for the data in (a) and save that chart

```
# Create the data for the chart.
v <- c(7,12,28,3,41)
# Give the chart file a name and plot the line chart.
png(file = "line_chart.jpg")
# Plot the bar chart.
plot(v,type = "o")
# Save the file.
dev.off()
```

c. Consider the data $v \leftarrow c(19, 23, 11, 5, 16, 21, 32,14, 19, 27, 39)$. Create a simple histogram for Range of X and Y values.

```
# Create data for the graph:
v <- c(19, 23, 11, 5, 16, 21, 32,14, 19, 27, 39)
# Create the histogram with x and y values:
hist(v, xlab = "No.of Articles ",col = "green", border = "black")
# Create the histogram.
hist(v, xlab = "No.of Articles", col = "green",border = "black", xlim = c(0, 50),ylim = c(0, 5), breaks = 5)
```

d. Create a histogram using non-uniform width

```
# Creating the histogram with non-uniform width:
hist(v, xlab = "Weight", ylab = "Frequency", xlim = c(50, 100), col = "darkmagenta", border = "pink", breaks = c(5, 55, 60, 70, 75,80, 100, 140))
```

11. Consider the dataset mtcars in R and perform the following.

- Write a R program to create the Box plot with chart file name
- Create a simple scatterplot to Plot the chart for cars with weight between 2.5 to 5 and mileage between 15 and 30.
- Create matrix of scatterplots for the relation between wt(weight),mpg(miles per gallon),disp(displacement) and cyl(number of cylinders). Save the chart.

Solution:

a. Write a R program to create the Box plot with chart file name

```
png(file = "boxplot.png")
# Plot the chart.
boxplot(mpg ~ cyl, data = mtcars, xlab = "Number of Cylinders", ylab = "Miles Per Gallon", main =
"Mileage Data")
# Save the file.
dev.off()
```

b. Create a simple scatterplot to Plot the chart for cars with weight between 2.5 to 5 and mileage between 15 and 30.

```
# Give the chart file a name.
png(file = "scatterplot.png")
# Plot the chart for cars with weight between 2.5 to 5 and mileage between 15 and 30.
plot(x = mtcars$wt, y = mtcars$mpg, xlab = "Weight", ylab = "Milage", xlim = c(2.5,5), ylim = c(15,30),
main = "Weight vs Milage" )
# Save the file.
dev.off()
```

c. Create matrix of scatterplots for the relation between wt(weight),mpg(miles per gallon),disp(displacement) and cyl(number of cylinders). Save the chart.

```
# Give the chart file a name.
png(file = "scatterplot_matrices.png")
# Plot the matrices between 4 variables giving 12 plots.
# One variable with 3 others and total 4 variables.
pairs(~wt+mpg+disp+cyl,data=mtcars,main="Scatterplot Matrix")
# Save the file.
dev.off()
```

12. Create R program and perform the following operations on them.

- a. Create a vector `x <- c(12,7,3,4.2,18,2,54,-21,8,-5)` and find mean .
- b. Apply trim option and NA option.
- c. Find the median for the vector `v <- c(2,1,2,3,1,2,3,4,1,5,5,3,2,3)` and calculate the mode for the character vector `charv <- c("o","it","the","it","it")` using the user function.

Solution:

a. Create a vector a and find mean

```
x <- c(12,7,3,4.2,18,2,54,-21,8,-5)
result.mean <- mean(x)
print(result.mean)
```

b. Apply trim option and NA option



```
result.mean <- mean(x,trim = 0.3)
print(result.mean)
```

```
# Create a vector.
x <- c(12,7,3,4.2,18,2,54,-21,8,-5,NA)
# Find mean.
result.mean <- mean(x)
print(result.mean)
# Find mean dropping NA values.
result.mean <- mean(x,na.rm = TRUE)
print(result.mean)
```

c. Find the median and Calculate the mode using the user function

```
median.result <- median(x)
print(median.result)
# To Calculate the mode Create the function.
find_mode <- function(x) {
  u <- unique(x)
  tab <- tabulate(match(x, u))
  u[tab == max(tab)]
}

# Create the vector with numbers.
v <- c(2,1,2,3,1,2,3,4,1,5,5,3,2,3)
# Calculate the mode using the user function.
result <- find_mode (v)
print(result)

# Create the vector with characters.
charv <- c("o","it","the","it","it")
# Calculate the mode using the user function.
result <- find_mode (charv)
print(result)
```

13. Create the data frame that contains student Details consisting of Student name, and marks for three subjects, namely, Physics, Chemistry and Mathematics. Perform the following analysis on it using dplyr library.
 - a. Use Select() function to print only column details starts with "Ph" and print everything that contains "cs"
 - b. Display data in which Mathematics marks lie between 90 and 94 using filter()
 - c. Display the physics mark on sorted order using arrange() function and rename the Chemistry column to CHEMISTRY using rename() function
 - d. Create the resultant data frame that contains new variable TotalMarks using mutate() function.

Solution:

- a. **Use Select() function to print only column details starts with "Ph" and print everything that contains "cs"**

```
# Create a data frame
df <- data.frame( Name = c("Bhuwanesh", "Anil", "Jai", "Naveen"),
  Physics = c(98, 87, 91, 94),
  Chemistry = c(93, 84, 93, 87),
  Mathematics = c(91, 86, 92, 83) )
print(df)
library(dplyr)
print(select(df, starts_with("Ph")))
print(select(df, contains("cs")))
```

- b. **Display data in which Mathematics marks lie between 90 and 94 using filter()**

```
# Display
print(df %>% filter(Mathematics > 90 & Mathematics < 94))
```

- c. **Display the physics mark on sorted order using arrange() function and rename the Chemistry column to CHEMISTRY using rename() function**

```
print(df %>% arrange(Physics))
# Display the dataset
# after renaming a column
```

```
print(df %>% rename(CHEMISTRY = Chemistry))
```

- d. **Create the resultant data frame that contains new variable TotalMarks using mutate() function.**

```
#df1<-mutate(df, TotalMarks = Physics + Chemistry+ Mathematics)
df1<-df %>% mutate(TotalMarks = Physics + Chemistry+ Mathematics)
print(df1)
```

14. Write a R program and perform following operations using tidyverse packages
 - a. use the map() function to calculate the square of each value in a vector c(2, 4, 10, 15, 20) and calculate mean value of each vector in list<- list(c(1, 2, 3), c(4, 5, 6), c(7, 8, NA)).
 - b. use str_replace() function to replaces a pattern or string with another string and use str_length() to find the length of each element of vector x<- c("R", "programming", "Language").
 - c. create the dataframe with letters and numbers and display the bar graph using ggplot()

Solution:

- a. use the map() function to calculate the square of each value in a vector c(2, 4, 10, 15, 20) and calculate mean value of each vector in list<- list(c(1, 2, 3), c(4, 5, 6), c(7, 8, NA))

```
library(purrr)
```

```
#define vector
```



NITTE
(Deemed to be University)

**NMAM INSTITUTE
OF TECHNOLOGY**

Nitte (DU) established under Section 3 of UGC Act 1956 | Accredited with 'A+' Grade by NAAC

```
data <- c(2, 4, 0, 15, 20)
#calculate square of each value in the vector
data %>%
  map(function(x) x^2)
```

```
library(purrr)
#define list of vectors
data <- list(c(1, 2, 3),
             c(4, 5, 6),
             c(7, 8, NA))
```

```
#calculate mean value of each vector in list
data %>%
  map(mean, na.rm=TRUE)
```

b. use `str_replace()` function to replaces a pattern or string with another string and use `str_length()` to find the length of each element of vector `x<- c("R", "programming", "Language")`.

```
library("stringr")
```

```
myString = "Bhuwanesh Nainwal"
```

```
# Replace if string starts with "Bhuwanesh"
print(str_replace(myString, "^Bhuwanesh", "Harshit"))
```

```
# Replace if string ends with "Nainwal"
print(str_replace(myString, "Nainwal$", ""))
```

```
x <- c("R", "programming", "Language")
print(str_length(x))
```

c. create the dataframe with letters and numbers and display the bar graph using ggplot()

```
library("ggplot2")
```

```
# create the dataframe with letters and numbers
df<-data.frame(x=c('A', 'B', 'C', 'D', 'E', 'F'),y=c(4, 6, 2, 9, 7, 3))
```

```
# display the bar
ggplot(df, aes(x, y, fill=x)) + geom_bar(stat="identity")
```

15. Create the data frame that contains revenue details of 4 years consisting of variables namely, Group, Year, Qtr.1 Qtr.2 Qtr.3 and Qtr.4. Perform the following analysis on it using tidyr library.
- Create a data frame by Re-structuring the time component Qtr.1 Qtr.2 Qtr.3 and Qtr.4 as an Quarter and Revenue using gather() function.
 - Use the result obtained from (a), Split the Quarter variable into Time_Interval and Interval_ID using separate().
 - Use the result obtained from (b) , re-unite the Time_Interval, Interval_ID variables and re-create the original Quarter variable using unite() function.
 - Use the result obtained from (c), reshape the long format to wide format using spread() function.

Solution:

- a. Create a data frame by Re-structuring the time component Qtr.1 Qtr.2 Qtr.3 and Qtr.4 as an Quarter and Revenue using gather() function.**

```
# Creating the data frame.
revenue_df <- data.frame(Group=c(1,1,1,1,2,2,2,2,3,3,3,3),
  Year=c(2006,2007,2008,2009,2006,2007,2008,2009,2006,2007,2008,2009),
  Qtr.1=c(15,12,22,10,12,16,13,23,11,13,17,14),
  Qtr.2=c(16,13,22,14,13,14,11,20,12,11,12,9),
  Qtr.3=c(19,27,24,20,25,21,29,26,22,27,23,31),
  Qtr.4= c(17,23,20,16,18,19,15,20,16,21,19,24))
# Printing the data frame.
print(revenue_df)
library(tidyr)
long_DF <- revenue_df %>% gather(Quarter, Revenue, Qtr.1:Qtr.4)
print(long_DF)
```

- b. Use the result obtained from (a), Split the Quarter variable into Time_Interval and Interval_ID using separate().**

```
separate_DF <- long_DF %>% separate(Quarter, c("Time_Interval", "Interval_ID"))
print(separate_DF)
```

- c. Use the result obtained from (b) , re-unite the Time_Interval, Interval_ID variables and re-create the original Quarter variable using unite() function.**

```
unite_DF <- separate_DF %>% unite(Quarter, Time_Interval, Interval_ID, sep = ".")
print(unite_DF)
```

- d. Use the result obtained from (c), reshape the long format to wide format using spread()**



function.

```
wide_DF <- unite_DF %>% spread(Quarter, Revenue)
print(wide_DF)
```

16. Consider mtcars dataset and perform the following statistical analysis

- Calculate the interquartile range, variance and Standard Deviation of "mpg" variable.
- Calculate covariance of the mpg and "wt" columns in the mtcars and create a covariance matrix of these two variables.
- Calculate the correlation between "mpg" and "wt".
- Implement regressor model to establish the relationship between "mpg" as a response variable with "dis", "hp" and "wt" as predictor variables. From the model, get the Coefficients and Create Equation for Regression Model and Apply Equation for predicting mileage for a car with disp = 221, hp = 102 and wt = 2.91

Solution:

- a. Calculate the interquartile range, variance and Standard Deviation of "mpg" variable.**

```
IQR(mtcars$ mpg)
var(mtcars$mpg) #using builtin function
#using formula
variance <- sum((mtcars$mpg - mean(mtcars$ mpg))^2) / (length(mtcars$ mpg) - 1)
sqrt(var(mtcars$mpg))
sd(mtcars$mpg)
```

- b. Calculate covariance of the "mpg" and "wt" columns in the mtcars and create a covariance matrix of these two variables.**

```
#using formula
sum((mtcars$mpg - mean(mtcars$mpg)) * (mtcars$wt - mean(mtcars$wt))) /
(length(mtcars$mpg) - 1)
#using builtin function
cov(mtcars$mpg, mtcars$wt)
#covariance matrix
library(dplyr)
mtcars %>% select(wt, mpg) %>% cov()
```

- c. Calculate the correlation between "mpg" and "wt".**

```
cor(mtcars$mpg, mtcars$wt)
cov(mtcars$mpg, mtcars$wt) / (sd(mtcars$mpg) * sd(mtcars$wt))
```

- d. Implement regressor model to establish the relationship between "mpg" as a response variable with "dis", "hp" and "wt" as predictor variables. From the model, get the Coefficients and Create Equation for Regression Model and Apply Equation for predicting mileage for a car with disp = 221, hp = 102 and wt = 2.91**

```
input <- mtcars[,c("mpg", "dis", "hp", "wt")]
# Create the relationship model.
model <- lm(mpg~dis+hp+wt, data=input)
# Show the model.
print(model)
# Get the Intercept and coefficients as vector elements.
```

```
cat("### The Coefficient Values ### ", "\n")
a <- coef(model)[1]
print(a)
Xdisp <- coef(model)[2]
Xhp <- coef(model)[3]
Xwt <- coef(model)[4]
print(Xdisp)
print(Xhp)
print(Xwt)
a <- data.frame(displacement = 221, horsepower = 102, weight = 2.91)
mpg <- predict(model,a)
print(mpg)
```